KubeQuest

Guide de Demo — Soutenance

Toutes les commandes a executer en live

⚠️ AVOIR SOUS LA MAIN : ce doc ouvert + 2 terminaux SSH sur kube-1 + navigateur avec les 4 URLs

---

AVANT LA SOUTENANCE — Checklist

⚠️ A faire 30 minutes avant la presentation. Verifier que tout tourne correctement.

1. Demarrer les 4 VMs sur AWS Console → EC2 → group-09
2. Recuperer les nouvelles IPs de kube-2/3/4 (sauf kube-1 : 13.36.145.234 fixe)
3. Se connecter en SSH sur kube-1 : ssh -i kubequest.pem ec2-user@13.36.145.234
4. Mettre a jour les ingress si l'IP de kube-2 a change (voir section 7)
5. Verifier l'etat du cluster (section 1 ci-dessous)
6. Ouvrir les 4 URLs dans le navigateur et verifier qu'elles repondent
7. Generer un token Dashboard frais

---

1. Verification Initiale du Cluster

A executer en PREMIER sur scene pour montrer que tout tourne.

1 Etat des noeuds

```
kubectl get nodes
# Resultat attendu :
# NAME                                        STATUS   ROLES
VERSION
```

```
# ip-10-0-9-232.eu-west-3.compute.internal   Ready    control-plane
v1.29.15
# ip-10-0-9-137.eu-west-3.compute.internal   Ready    <none>
v1.29.15
```

2 Tous les pods en Running

```
kubectl get pods -A | grep -v Completed
# Tous les pods doivent etre en Running ou 1/1, 2/2, 3/3...
```

3 Ingress et URLs

```
kubectl get ingress -A
# NAMESPACE    NAME           HOSTS
# monitoring   grafana        grafana.35.180.92.133.nip.io
# sample-app   sample-app     app.35.180.92.133.nip.io
# argocd       argocd         argocd.35.180.92.133.nip.io
# ...
```

4 Application accessible

```
curl -s -o /dev/null -w "%{http_code}"
http://app.35.180.92.133.nip.io/
# Resultat attendu : 200
```

2. Demo Auto-Scaling HPA

Montre que le cluster s'adapte automatiquement a la charge.

⚠️ Ouvrir 2 terminaux SSH sur kube-1. Terminal 1 = surveillance. Terminal 2 = envoi de charge.

1 Terminal 1 — Surveiller le HPA

```
kubectl get hpa -n sample-app -w
# AVANT la charge :
# NAME         REFERENCE               TARGETS   MINPODS   MAXPODS
REPLICAS
# sample-app   Deployment/sample-app   30%/70%   2         5         2
```

2 Terminal 2 — Envoyer la charge

```
for i in $(seq 1 2000); do
  curl -s -o /dev/null http://app.35.180.92.133.nip.io/ &
  if (( i % 50 == 0 )); then wait; fi
done
echo 'Charge terminee !'
```

3 Observer dans Terminal 1

```
# Le CPU va monter au dessus de 70%
# sample-app   Deployment/sample-app   186%/70%   2   5   2
# sample-app   Deployment/sample-app   251%/70%   2   5   4
# sample-app   Deployment/sample-app   83%/70%    2   5   5
```

4 Verifier les nouveaux pods

```
kubectl get pods -n sample-app
# Resultat attendu : 5 pods sample-app en Running
# NAME                          READY   STATUS    RESTARTS
# sample-app-xxx-1              1/1     Running   ...
# sample-app-xxx-2              1/1     Running   ...
# sample-app-xxx-3              1/1     Running   ...
# sample-app-xxx-4              1/1     Running   ...
# sample-app-xxx-5              1/1     Running   ...
```

✅ Demo HPA reussie : 2 → 5 replicas automatiquement. Le cooldown de 5 minutes fera revenir a 2.

3. Demo Rollback Automatique

Montre la resilience : deploiement casse → detection → retour automatique a la version stable.

1 Lancer le script de demo

```
cd ~/T-CLO-902-NCE_7
bash scripts/demo/broken-deploy.sh
```

2 Ce que le script fait automatiquement

- Deploie une image cassee (inexistante)
- Surveille le rollout — detecte l'echec (ImagePullBackOff)
- Execute helm rollback automatiquement
- Verifie que la version stable est restauree

3 Resultat attendu

```
# Le script affiche :
[INFO] Deploiement d'une image cassee...
[INFO] Attente du rollout...
[WARN] Rollout echoue - declenchement du rollback...
[INFO] Rollback en cours...
deployment.apps/sample-app rolled back
[OK] Demo terminee. Image restauree: hanenetriaa/sample-app:latest
```

✅ Demo Rollback reussie : retour a la version stable en moins de 30 secondes.

---

4. Demo Backup Base de Donnees

Montre le systeme de sauvegarde automatique MySQL.

1 Voir le CronJob planifie

```
kubectl get cronjob -n sample-app
# NAME                     SCHEDULE    LAST SCHEDULE
# sample-app-mysql-backup   0 2 * * *   77m
```

2 Declencher un backup manuel

```
JOB_NAME="backup-demo-$(date +%H%M%S)"
kubectl create job --from=cronjob/sample-app-mysql-backup ${JOB_NAME}
-n sample-app
kubectl wait --for=condition=complete job/${JOB_NAME} -n sample-app
--timeout=180s
```

3 Verifier que le backup est complete

```
kubectl get jobs -n sample-app
# NAME              COMPLETIONS   DURATION
# backup-demo-xxx   1/1           9s
```

✅ Demo Backup reussie : dump MySQL complete en 9 secondes, stocke dans le PVC dedie.

5. Demo ArgoCD GitOps

Montre la synchronisation automatique depuis GitHub.

1 Ouvrir ArgoCD dans le navigateur

```
URL      : https://argocd.35.180.92.133.nip.io
Login    : admin
Password: jyIyop58jwbGldPS
```

2 Montrer l'application synchronisee

- Cliquer sur l'application sample-app
- Montrer : Status = Healthy + Synced
- Montrer : Path = gitops/overlays/production
- Montrer : le dernier commit GitHub synchronise

3 Commandes de verification

```
# Verifier le status ArgoCD en CLI
kubectl get applications -n argocd
# NAME         SYNC STATUS   HEALTH STATUS
# sample-app   Synced        Healthy
```

✅ Demo ArgoCD reussie : cluster synchronise avec GitHub automatiquement.

6. Demo Monitoring Grafana

Montre l'observabilite de la plateforme.

1 Ouvrir Grafana

```
URL      : http://grafana.35.180.92.133.nip.io
Login    : admin
Password : prom-operator
```

2 Dashboards a montrer

- Kubernetes / Compute Resources / Cluster — vue globale CPU/RAM
- Kubernetes / Compute Resources / Namespace (sample-app) — metriques de l'app
- Kubernetes / Compute Resources / Pod — detail par pod

3 Montrer l'impact du test de charge

Apres la demo HPA, montrer dans Grafana la montee en CPU et le scale-out des pods.

7. Si l'IP de kube-2 a Change

A faire si les URLs nip.io ne repondent plus apres redemarrage.

⚠️ Recuperer la nouvelle IP de kube-2 sur AWS Console → EC2 → ec2-group-09-node-2 → IPv4 publique

```
# Remplacer NEW_IP par la nouvelle IP de kube-2
NEW_IP=<nouvelle-ip>

# Mettre a jour tous les ingress
kubectl patch ingress sample-app -n sample-app \
  --type='json' \

-p="[{\"op\":\"replace\",\"path\":\"/spec/rules/0/host\",\"value\":\"a
pp.${NEW_IP}.nip.io\"}]"
```

```
kubectl patch ingress grafana -n monitoring \
  --type='json' \

-p="[{\"op\":\"replace\",\"path\":\"/spec/rules/0/host\",\"value\":\"g
rafana.${NEW_IP}.nip.io\"}]"

kubectl patch ingress kubernetes-dashboard -n kubernetes-dashboard \
  --type='json' \

-p="[{\"op\":\"replace\",\"path\":\"/spec/rules/0/host\",\"value\":\"d
ashboard.${NEW_IP}.nip.io\"}]"

kubectl patch ingress argocd -n argocd \
  --type='json' \

-p="[{\"op\":\"replace\",\"path\":\"/spec/rules/0/host\",\"value\":\"a
rgocd.${NEW_IP}.nip.io\"}]"
```

8. Commandes Rapides de Reference

Token Dashboard

```
kubectl -n kubernetes-dashboard create token admin-user
```

Logs d'un pod

```
kubectl logs <pod-name> -n <namespace> -f
```

Etat complet du cluster

```
kubectl get pods -A
kubectl get svc -A
kubectl get ingress -A
kubectl get hpa -A
kubectl get pvc -A
```

Redemarrer un deploiement

```
kubectl rollout restart deployment/sample-app -n sample-app
```

Deploiement complet depuis zero

```
cd ~/T-CLO-902-NCE_7
bash deploy-all.sh
```

Status Helm releases

```
helm list -A
```